Allocating Bits Between Coordinate Pairs

A real signal before SAQ rotation, but no usable opportunity after it

2026-09-04

1. The Answer First

Final decision

Stop the current pair-level allocation direction. Do not build its query consumer.

- Before SAQ's internal rotation, moving bits between coordinate pairs looks valuable: about 28–31% on SIFT and 6.3–6.4% on GIST residuals.
- After the rotation used inside each SAQ segment, every pair chooses the bit width it already had.
- At exactly matched payload and factor bytes, the final measured direction-error improvement is **0%**.

Main lesson

The earlier positive result was not numerically wrong. It measured structure that a later SAQ step removes before quantization and search.

2. Correction: What Was the Intended Question?

Earlier interpretation

For one pair (D_1, D_2), decide how many states belong to D_1 and how many belong to D_2.

The intended question

Treat (D_1, D_2) as one unit and (D_3, D_4) as another. Decide how much of the total storage each *pair* receives.

Why this matters

This is allocation **between coordinate pairs**, not allocation between the two coordinates inside one pair. The September branch was created to test this corrected question directly.

3. Running Example: Give Bits to the Harder Pair

Assume two coordinate pairs share a total budget of 16 bits.

Pair	Variation in the data	Equal allocation	Flexible allocation
(D_1, D_2)	large	8 bits	10 bits
(D_3, D_4)	small	8 bits	6 bits
Total		16 bits	16 bits

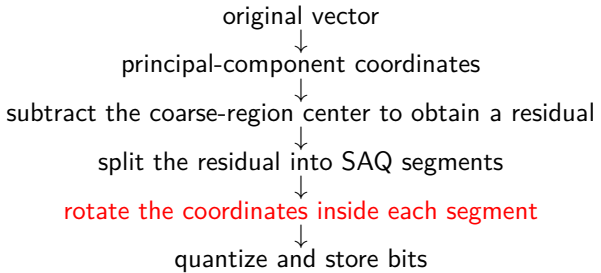
Intuition

An extra bit creates more reconstruction levels. It should go to the pair where those extra levels reduce more error. The easy pair gives up two bits; the hard pair receives them; total storage does not change.

Testable hypothesis

If different pairs have persistently different returns from an extra bit, flexible allocation should lower held-out error by at least 5%.

4. Where Does This Happen in the SAQ Pipeline?



Two places to measure

The first diagnostic measured pairs before the segment rotation. The decisive check measured the coordinates that the SAQ quantizer actually receives, after that rotation.

A residual is simply the vector left after subtracting the center of its coarse database region.

5. First Diagnostic: How We Allocated the Budget

```
Use 16,384 fixed base vectors; split them into halves A and B.

For each representation stage and each two-coordinate group:
  fit the local quantizer on A
  measure error for candidate group budgets from 6 to 10 bits

Use exact dynamic programming:
  choose one budget for every group
  keep the total at exactly 512 bits
  minimize total fitting error

Evaluate the frozen allocation on B; then swap A and B.
```

No query tuning

Only learn/base vectors were used. Benchmark queries, exact nearest-neighbour answers, Recall, and query speed were not read.

6. The Signal Survived PCA and Coarse Residualization

Held-out reconstruction-error reduction from allocating bits between fixed adjacent pairs:

Dataset	principal-component view	1,024 coarse regions	4,096 coarse regions
SIFT1M	40.59%	22.55%	20.11%
GIST1M	30.82%	13.93%	11.73%

What this established

Unequal demand between two-coordinate groups is not merely an artifact of the original raw coordinates. It remains visible after the main representation steps used to prepare SAQ residuals.

What this did not establish

The comparison was against equal 8-bit group budgets, not yet against the complete SAQ representation.

7. Correlation Was Not the Main Explanation

For each residual group, we compared the value of one extra bit with two simple statistics.

Total variation of the pair	correlation with extra-bit value was 0.966–0.995
Absolute correlation between the coordinates	only 0.041–0.089 on GIST and 0.246–0.306 on SIFT

Interpretation

The useful rule was essentially "give more bits to groups containing more energy." It was not "pair the most strongly correlated coordinates."

This is important because ordinary rate–distortion allocation already uses the first principle.

8. Closest Work Already Covers the General Principle

Transform Coding (2010)	principal-component transform, trained scalar quantizers, and unequal bit allocation under one total budget
Optimized Transform Coding (2014)	component-specific error estimates for choosing the bit allocation
OPQ (2013)	optimized rotation and decomposition for product quantization
Adaptive Bit Allocation PQ (2016)	unequal codebook sizes between product- quantizer subspaces
Distribution-Sensitive PQ (2018)	moves bits between subspaces according to their distributions
SAQ (2026)	dynamic programming for segment boundaries and one bit width per segment under a storage budget

Novelty decision

Local 2D rotation + fitted error curves + exact budget DP is a direct composition of known ideas. Exact DP improves correctness; it does not create a new quantization model.

9. The Narrow SAQ-Specific Question

What might still have been new

Inside one existing SAQ segment, let different coordinate pairs store different numbers of lower bits, while preserving SAQ's shared scale and two-stage scan.

The test had to preserve all of the following:

- the same segment boundaries and number of segment factors;
- exactly the same total payload and factor bytes;
- one common scale for the whole vector segment;
- one fast bit for every represented coordinate; and
- no per-vector plan identifier or per-pair factor.

Why test before implementation?

A ragged-bit query consumer would add packing, decoding, and branching work. We first asked whether enough base-only quality opportunity remained to justify that engineering.

10. The Exact Storage-Matched Baselines

Dataset	Existing SAQ plan	Total counted storage
SIFT1M	all 128 dimensions use 4 bits each	72 bytes/vector
GIST1M	successive groups of 64, 192, 320, 256, and 128 dimensions use 11, 6, 4, 2, and 0 bits each	488 bytes/vector

Why factors are counted

Every positive SAQ segment stores two additional floating-point values per database vector. Allowing more segments would secretly spend more bytes, so we kept the existing segments and factor count unchanged.

Only widths *inside* each positive segment could move between adjacent pairs.

11. Running Example: Rotation Removes the Imbalance

Consider four independent coordinates grouped into two pairs:

$$\underbrace{(D_1, D_2)}_{\text{variances } 9,9}, \qquad \underbrace{(D_3, D_4)}_{\text{variances } 1,1}.$$

Before rotation, the first pair contains 90% of the total variation, so it clearly deserves more bits.

A balanced orthogonal rotation mixes all four coordinates without losing information. In this simple example, it can spread the same total variation as

$$(5, 5, 5, 5).$$

After rotation

Both new pairs now have equal variation. Moving a bit from one pair to the other only makes one side worse; the equal-width allocation is optimal again.

SAQ performs this kind of mixing separately inside every segment.

12. Decisive Opportunity Decomposition

```
Recover the current SAQ plan from fitting-half variances.

For every positive segment:
  keep its dimensions, payload bits, and two stored factors
  apply one fixed and recorded orthogonal rotation
  fit actual uniform-lattice error curves for every adjacent pair
  reallocate pair widths with exact dynamic programming

For held-out vectors:
  use the segment-wide maximum and shared scale
  run the same six rounds of coordinate adjustment
  compare direction-related error with the uniform segment baseline

Swap the fitting and evaluation halves and repeat.
```

Frozen stopping rule

Stop if the post-rotation gain is below 5% on either natural dataset.

13. Before Rotation: The Opportunity Still Looks Large

Held-out improvement predicted by SAQ's own variance-based planning formula:

Dataset and residual view	A fitted, B tested	B fitted, A tested
SIFT, 1,024 regions	30.74%	30.83%
SIFT, 4,096 regions	28.20%	28.39%
GIST, 1,024 regions	6.45%	6.39%
GIST, 4,096 regions	6.39%	6.31%

This is an upper-level diagnostic, not the answer

These coordinates have not yet passed through the rotation immediately before SAQ quantization. Using this table alone would repeat the earlier mistake.

14. After Rotation: Every Pair Returns to Baseline

Dataset	residual settings	pairs changing width	final error gain
SIFT1M	2 region counts $\times$ 2 folds	**0**	**0%**
GIST1M	2 region counts $\times$ 2 folds	**0**	**0%**

- Every SIFT pair selects the existing 4-bit width.
- Every positive GIST pair selects its segment's existing 11-, 6-, 4-, or 2-bit width.
- The variance formula and the fitted uniform-lattice curves agree.
- With identical selected widths, the adjusted database codes and their measured error are identical to the baseline.

Gate result

The required 5% opportunity is missed by the full 5 percentage points.

15. Why the Two Positive Tables Were Not Contradictory

Before segment rotation

PCA orders coordinates from high to low importance. Adjacent pairs inherit large differences, so bit allocation helps.

After segment rotation

SAQ deliberately mixes coordinates inside each segment. Pair-level demand is equalized, so the same allocation no longer helps.

Research interpretation

The signal exists at one stage of the pipeline but is not available to the unchanged SAQ representation and consumer. Representation-stage attribution prevented us from building the wrong system.

16. What Is Closed, and What Is Not?

Closed by this result

- keep the current SAQ segment rotation;
- reallocate bits between adjacent pairs afterward;
- build a ragged mixed-width consumer for that allocation.

Not proved impossible

- every possible orthogonal rotation;
- jointly redesigning transform and allocation;
- unrelated SAQ-specific limitations.

Important novelty boundary

A joint transform/allocation method would be a new research question under strong Transform Coding, OPQ, BAPQ, and DSPQ prior-work pressure. It is not a positive continuation supported by this experiment.

17. Evidence Quality and Cost

- Official TexMex SIFT1M and GIST1M learn/base files were hash verified.
- No benchmark query, exact-answer file, Recall result, QPS result, or old ANN index result was read.
- The two data halves were used in both fit/evaluation directions.
- Ten focused tests pass; generalized mixed-width adjustment matches an independent scalar implementation on a tiny fixture.
- Two accepted executions produced byte-identical metadata and complete scientific summaries.
- Each accepted run used about 94 CPU-seconds and less than 716 MB peak memory.

Performance status

This was an early base-only falsification test, not Recall or query-speed evidence. Because the quality opportunity is zero, production optimization is not justified.

18. Final Takeaway

Decision

Stop the current post-rotation pair-allocation direction.

1. The corrected pair-level question was tested directly.
2. Unequal group demand is real before the final SAQ segment rotation.
3. Variance, not within-pair correlation, explains most of the signal.
4. Existing work already covers the general allocation principle.
5. Under the unchanged SAQ representation and exact byte budget, the remaining usable opportunity is 0%.

Next research step

Close this branch. Start a new direction only from a limitation specific to SAQ's shared factors, adjustment rule, or two-stage query consumer, rather than trying to rescue this allocation result.

References

- Brandt. *Transform Coding for Fast Approximate Nearest Neighbor Search in High Dimensions*. CVPR, 2010.
- Ge et al. *Optimized Product Quantization*. CVPR, 2013.
- Park et al. *Optimized Transform Coding for Approximate KNN Search*. BMVC, 2014.
- Guo et al. *Adaptive Bit Allocation Product Quantization*. Neurocomputing, 2016.
- Li et al. *Distribution Sensitive Product Quantization*. IEEE TCSVT, 2018.
- Andre et al. *Quicker ADC: Unlocking the Hidden Potential of Product Quantization with SIMD*. IEEE TPAMI, 2021.
- Li et al. *Scalable Accurate Quantization for Fast Similarity Search*. SIGMOD, 2026.

Detailed project evidence: representation-stage attribution, closest-primary-work review, and matched-byte opportunity decomposition dated 2026-09-03 in `docs/research/`.